

ReconForge Workflow Orchestration Guide

Author: Andrews Ferreira

Overview

The **Workflow Orchestrator** is the brain of ReconForge's cross-module automation. Instead of running modules one at a time, you define a pipeline of steps—each mapped to a module—and the orchestrator:

1. Executes them in order.
 2. Passes a shared **WorkflowContext** between steps (discovered hosts, ports, domains, URLs).
 3. Evaluates **conditions** before each step so modules only run when their prerequisites are met (e.g., AD recon only fires when LDAP or Kerberos is detected).
 4. Shares credentials across modules through the **CredentialVault**.
 5. Optionally tracks the entire session through the **EngagementManager**.
-

Quick Start

Full Reconnaissance (all modules, conditional)

```
python reconforge.py workflow --target 10.10.10.1 --opsec normal
```

This builds the default pipeline:

```
surface → network → ad (if LDAP/Kerberos found) → web (if HTTP found) → api (if HTTP found)
```

Targeted Modules

```
python reconforge.py workflow --target 10.10.10.1 \  
  --modules network,web --opsec stealth
```

With Engagement Tracking

```
python reconforge.py workflow --target 10.10.10.1 \  
  --engagement "Acme Assessment" --client "Acme Corp" --operator "alice"
```

Workflow Orchestrator Flow Diagram

```

flowchart TD
    START(["WorkflowOrchestrator.run()"])
    INIT["Initialize WorkflowContext<br/>ctx.targets = targets"]
    ENG{"engagement.status<br/>== 'planning'?"}
    ENG_START["engagement.start()<br/>Set scope = targets"]
    LOOP["For each WorkflowStep<br/>i = 1..N"]
    HAS_COND{"step.condition<br/>is not None?"}
    EVAL_COND{"condition(ctx)<br/>returns True?"}
    SKIP["step_result.status = 'skipped'<br/>engagement.record_action(step_skipped)"]
    PICK["_pick_target(step)<br/>Select best target"]
    RUN_MOD["_run_module(module_name, target)<br/>Dynamic import + execute"]
    EXTRACT["_extract_context_from_result()<br/>Populate ctx: hosts, ports,<br/>>services, domains, urls"]
    STORE["ctx.store_result(module, result)<br/>engagement.record_module_result()"]
    SUCCESS["step_result.status = 'success'"]
    FAIL{"Exception<br/>raised?"}
    CRITICAL{"step.critical<br/>== True?"}
    ABORT["status = 'failed'<br/>Abort workflow"]
    NONCRIT["status = 'failed'<br/>Continue to next step"]
    MORE{"More steps<br/>remaining?"}
    POST["engagement.complete()<br/>_build_summary()<br/>_save_workflow_report()"]
    DONE(["Return summary dict"])

    START --> INIT --> ENG
    ENG -- Yes --> ENG_START --> LOOP
    ENG -- No --> LOOP
    LOOP --> HAS_COND
    HAS_COND -- No --> PICK
    HAS_COND -- Yes --> EVAL_COND
    EVAL_COND -- No --> SKIP --> MORE
    EVAL_COND -- Yes --> PICK
    PICK --> RUN_MOD --> FAIL
    FAIL -- No --> EXTRACT --> STORE --> SUCCESS --> MORE
    FAIL -- Yes --> CRITICAL
    CRITICAL -- Yes --> ABORT --> POST
    CRITICAL -- No --> NONCRIT --> MORE
    MORE -- Yes --> LOOP
    MORE -- No --> POST --> DONE
  
```

Default Full-Recon Pipeline (add_full_recon)

```

flowchart LR
    S["Surface<br/><i>condition: host_count > 0</i>"]
    N["Network<br/><i>no condition (always runs)</i>"]
    AD["AD<br/><i>condition: ldap v ldaps v<br/>kerberos v smb v msrpc<br/>v has_domains</i>"]
    W["Web<br/><i>condition: http v https v<br/>http-alt v https-alt<br/>v has_url</i>"]
    API["API<br/><i>condition: http v https v<br/>http-alt v https-alt<br/>v has_url</i>"]

    S -->|"ctx: ports, services"| N
    N -->|"ctx: hosts, ports,<br/>services, domains"| AD
    AD -->|"ctx: domains"| W
    W -->|"ctx: urls"| API
  
```

Core Concepts

WorkflowContext

A shared data bus that flows through every step.

Attribute	Type	Description
targets	List[str]	Original target list
live_hosts	List[str]	Hosts confirmed alive
open_ports	Dict[str, List[int]]	Per-host open ports
services	Dict[str, Set[str]]	Per-host service names
domains	List[str]	Discovered AD/DNS domains
urls	List[str]	Discovered web URLs
module_results	Dict[str, Dict]	Raw result dicts from each module
extra	Dict[str, Any]	Free-form storage for custom data

Helper methods:

```
ctx.has_service("ldap") # True if any host exposes LDAP
ctx.has_port(443)       # True if any host has 443 open
ctx.has_domain()        # True if AD domain was discovered
ctx.has_url()           # True if web URLs were found
ctx.host_count()        # Number of live hosts (or targets)
```

CredentialVault

Centralised, deduplicated credential storage shared across all modules.

```

from core.credential_vault import CredentialVault

vault = CredentialVault()
vault.add_password("admin", "P@ssw0rd", source="hydra", module="network")
vault.add_hash("aad3b435...", "NTLM", username="admin", module="ad")
vault.add_token("eyJhbG...", "jwt", source="burp", module="web")
vault.add_api_key("AKIA...", source="trufflehog", module="api")
vault.add_ssh_key("-----BEGIN RSA...", username="root")
vault.add_username("jsmith", source="ldap", module="ad")

# Query
vault.get_passwords()
vault.get_hashes()
vault.get_for_service("ssh")
vault.get_for_module("ad")
vault.get_usernames()

# Loot integration
vault.ingest_from_loot(loot_manager) # auto-import from LootManager
vault.contribute_to_loot(loot_manager) # push vault creds into LootManager

# Persistence
vault.save(Path("vault.json"))
vault.load(Path("vault.json"))
vault.export_usernames(Path("users.txt"))

```

Deduplication is fingerprint-based: `type|username|secret|domain|service`.

Adding the same credential twice is silently ignored.

EngagementManager

Lifecycle tracking for professional engagements.

```

from core.engagement import EngagementManager

eng = EngagementManager(client="Acme Corp", operator="alice")
eng.start() # planning → active
eng.record_action("network", "port_scan", "TCP SYN top 1000")
eng.record_module_result("network", results)
eng.pause() # active → paused
eng.resume() # paused → active
eng.complete() # active → completed

eng.save(Path("engagement.json"))
eng2 = EngagementManager.load(Path("engagement.json"))

```

Valid state transitions:

```

planning → active → paused → active → completed
           → cancelled

```

Conditions

Conditions are callables `(WorkflowContext) → bool`. Built-in conditions:

Condition	Fires when ...
<code>condition_ad</code>	LDAP (389/636) or Kerberos (88) detected
<code>condition_web</code>	HTTP (80/443/8080/8443) detected
<code>condition_api</code>	HTTP detected (same as web)
<code>condition_surface</code>	Always <code>True</code>

Custom Conditions

```
from core.workflow_orchestrator import WorkflowOrchestrator

wo = WorkflowOrchestrator(targets=["10.10.10.1"])
wo.add_step("ad", condition=lambda ctx: ctx.has_service("ldap"))
wo.add_step("web", condition=lambda ctx: ctx.has_port(443))
wo.run()
```

Programmatic Usage

Minimal Pipeline

```
from core.workflow_orchestrator import WorkflowOrchestrator

wf = WorkflowOrchestrator(targets=["10.10.10.1"])
wf.add_step("network")
wf.add_step("web", condition=lambda ctx: ctx.has_port(80))
wf.run()
```

Full Recon (class method)

```
wf = WorkflowOrchestrator.full_recon(
    ["10.10.10.1"],
    opsec_mode="stealth",
    verbose=True,
)
wf.run()
```

Targeted (class method)

```
wf = WorkflowOrchestrator.targeted(
    ["10.10.10.1"],
    modules=["network", "ad"],
    opsec_mode="normal",
)
wf.run()
```

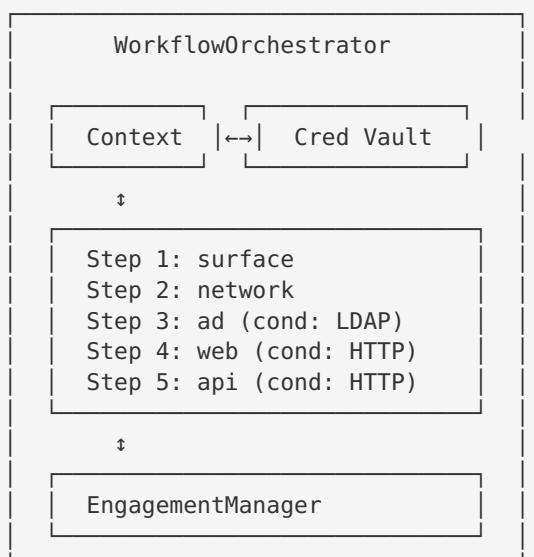
With Engagement

```
from core.engagement import EngagementManager

eng = EngagementManager(client="Acme", operator="alice")
wf = WorkflowOrchestrator(
    targets=["10.10.10.1"],
    engagement=eng,
    opsec_mode="normal",
)
wf.add_full_recon()
wf.run()

eng.save(Path("outputs/engagement.json"))
```

Architecture



Data flow per step:

1. Evaluate condition against `WorkflowContext` .
2. If `True` , instantiate the module.
3. Inject `CredentialVault` into the module.
4. Run the module → collect results.
5. Extract context (hosts, ports, services, domains, URLs) from results.
6. Store results in `WorkflowContext.module_results` .
7. Ingest new credentials from module's `LootManager` into the vault.
8. Record action in `EngagementManager` (if enabled).

Input Validation

The `core.validators` module provides reusable validation helpers:

```

from core.validators import (
    validate_ip, validate_cidr, validate_hostname,
    validate_target, validate_port, validate_port_range,
    parse_port_list, validate_url, validate_domain,
)

validate_ip("10.0.0.1")           # "10.0.0.1"
validate_cidr("10.0.0.0/24")      # "10.0.0.0/24"
validate_port("443")              # 443
parse_port_list("22,80-82,443")   # [22, 80, 81, 82, 443]
validate_url("https://app.com")   # "https://app.com"

```

All validators raise `ValidationError` (from `core.exceptions`) on invalid input.

Custom Exceptions

All workflow-related errors inherit from `ReconForgeError` :

Exception	When raised
<code>WorkflowError</code>	Pipeline misconfiguration or step failure
<code>CredentialVaultError</code>	Vault encryption/decryption issues
<code>EngagementError</code>	Invalid lifecycle state transitions
<code>ValidationError</code>	Input validation failures
<code>ExecutionError</code>	Tool execution failures
<code>ToolNotFoundError</code>	Required external tool not installed
<code>TimeoutError</code>	Tool execution exceeds timeout

CLI Reference

```
python reconforge.py workflow [OPTIONS]
```

Options:

--target TARGET	Target IP, hostname, or CIDR
--modules MODULES	Comma-separated module list (default: full recon)
--opsec MODE	OPSEC mode: stealth normal aggressive
--engagement NAME	Engagement name for tracking
--client NAME	Client name for engagement
--operator NAME	Operator name for engagement
--resume FILE	Resume from saved engagement JSON
-v, --verbose	Verbose output
--dry-run	Show commands without executing
--timeout SECONDS	Per-tool timeout (default: 600)
--encrypt-loot	Encrypt loot/vault files
-o, --output DIR	Output base directory